MANUAL DE TECNICO (PRUEBA DIAGNÓSTICA)

ANDRES FELIPE TAPIAS MENA

HELEN JANINA QUEVEDO POLO

SEMILLERO DE INVESTIGACIÓN GIDSAW

UNIVERSIDAD DE INVESTIGACIÓN Y DESARROLLO (UDI)

BUCARAMANGA SANTANDER

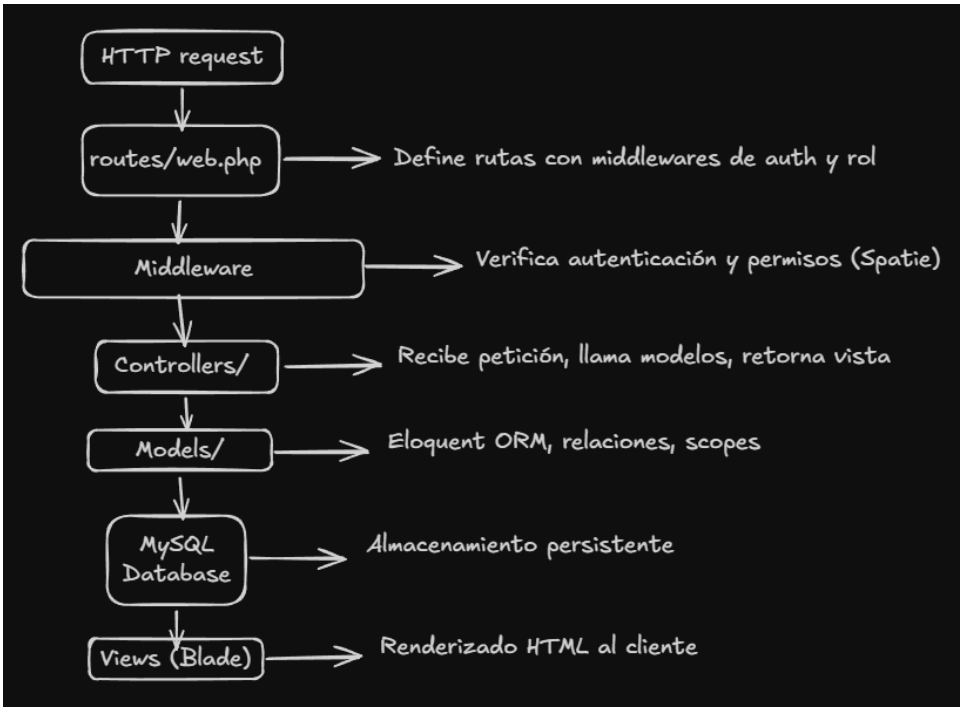
2026

1. Información general

Ítem	Detalle
Framework	Laravel 12
PHP	8.2+
Base de datos	MySQL 8.0+
Autenticación	Laravel Breeze
Roles y permisos	spatie/laravel-permission ^6.0
Frontend	Blade + Tailwind CSS
Pruebas	PHPUnit (37 pruebas)

2. Arquitectura del sistema

El proyecto sigue el patrón **MVC (Modelo-Vista-Controlador)** de Laravel con una capa adicional de servicios para mantener los controladores limpios.



3. Estructura de carpetas

- **gestion-tienda/**
  - **app/** (Lógica del núcleo)
    - **Http/Controllers/**
      - `Auth/` (Breeze)
      - `DashboardController.php` (Lógica por rol)
      - `ProductController.php` (CRUD)
      - `UserController.php` (Gestión)
    - **Http/Middleware/**
      - `RoleMiddleware.php`
    - **Models/**
      - `User.php` (Spatie HasRoles)
      - `Product.php` (Scopes de búsqueda)
  - **database/** (Persistencia)
    - `migrations/` (Users, Products, Roles)
    - `seeders/` (RoleSeeder, UserSeeder, ProductSeeder)
    - `factories/`
  - **resources/views/** (Interfaz)
    - `layouts/app.blade.php` (Layout principal)
    - `dashboard/` (Vistas admin/empleado)
    - `products/` (CRUD y Form partials)
    - `users/`
    - `errors/403.blade.php`
  - **tests/** (Calidad)
    - `Unit/ProductTest.php`
    - `Feature/` (Auth, Roles, CRUD)
  - `routes/web.php` (Definición de rutas y grupos)
  - `docs/` (Documentación adicional)
  - `nixpacks.toml` (Configuración Railway)

4. Modelos y relaciones

User
// Traits
use HasRoles; // Spatie: assignRole(), hasRole(), getRoleNames()

// Relaciones
public function products(): HasMany // Un usuario crea muchos productos

Product
// Campos fillable
['name', 'description', 'price', 'stock', 'category', 'image', 'user_id']

// Scopes
public function scopeSearch($query, string $term) // Busca en name y description
public function scopeByCategory($query, string $cat) // Filtra por categoría

// Métodos
public function inStock(): bool // Retorna true si stock > 0

// Relaciones
public function user(): BelongsTo // Pertenece a un usuario

5. Roles y permisos

Roles definidos

Rol	Descripción
admin	Acceso total al sistema
empleado	Acceso limitado a productos

Permisos por rol

Permiso	Admin	Empleado
ver productos	SI	SI
crear productos	SI	SI
editar productos	SI	SI
eliminar productos	SI	NO
ver usuarios	SI	NO
crear usuarios	SI	NO
asignar roles	SI	NO

Implementación en rutas

// Solo usuarios autenticados
```
Route::middleware(['auth'])->group(function () {

 // Admin y Empleado
   Route::middleware(['role:admin|empleado'])->group(function () {
      Route::resource('products', ProductController::class)->except(['destroy']);
   });

 // Solo Admin
   Route::delete('/products/{product}', ...)->middleware('role:admin');
   Route::resource('users', UserController::class)->middleware('role:admin');
```

```
});
```

Verificación en vistas Blade

```
@role('admin')
    {{-- Solo visible para Admin --}}
@endrole

@can('eliminar productos')
    {{-- Solo si tiene el permiso --}}
@endcan
```

6. Controladores

DashboardController

Decide qué vista mostrar según el rol del usuario autenticado.

```
public function index()   // Redirige internamente a adminDashboard() o empleadoDashboard()
```

ProductController

CRUD completo con manejo de imágenes en storage/app/public/products/.

```
public function index(Request $request)          // Lista con búsqueda y filtros
public function create()                         // Formulario crear
public function store(Request $request)          // Guardar nuevo producto
public function show(Product $product)           // Ver detalle
public function edit(Product $product)           // Formulario editar
public function update(Request $request, ...)    // Actualizar producto
public function destroy(Product $product)        // Eliminar + borrar imagen
```

UserController

Solo accesible para Admin. Incluye protecciones contra auto-eliminación.

```
public function index()                          // Lista paginada de usuarios
public function create()                         // Formulario crear usuario
public function store(Request $request)          // Guardar usuario + asignar rol
public function edit(User $user)                 // Formulario editar
public function update(Request $request, ...)    // Actualizar + sincronizar rol
public function destroy(User $user)              // Eliminar con validaciones
```

7. Validaciones implementadas

Productos

Campo	Regla
name	required, string, max:255
price	required, numeric, min:0.01
stock	required, integer, min:0
category	required, string, max:100
image	nullable, image, mimes:jpg,jpeg,png,webp, max:2048

Usuarios

Campo	Regla
name	required, string, max:255
email	required, email, unique:users
password	required, string, min:8, confirmed
role	required, exists:roles,name

8. Manejo de imágenes

Las imágenes se almacenan en storage/app/public/products/ y son accesibles públicamente mediante el enlace simbólico creado con php artisan storage:link.

```
// Guardar imagen
$path = $request->file('image')->store('products', 'public');

// Acceder en vista
Storage::url($product->image)

// Eliminar imagen
Storage::disk('public')->delete($product->image);
```

9. Configuración de pruebas

Las pruebas usan **SQLite en memoria** para no afectar la base de datos de producción.

```
<!-- phpunit.xml -->
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
```

Cada prueba que necesita roles ejecuta:

```
$this->seed(\Database\Seeders\RoleSeeder::class);
```

10. Comandos útiles

```
# Migraciones
php artisan migrate                    # Correr migraciones
php artisan migrate:fresh --seed       # Resetear BD y poblar
php artisan migrate:rollback           # Revertir última migración

# Caché
php artisan config:clear
php artisan route:clear
php artisan view:clear
php artisan cache:clear

# Optimización (producción)
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan optimize

# Pruebas
php artisan test
php artisan test --verbose
php artisan test --filter NombrePrueba

# Tinker (consola interactiva)
php artisan tinker

# Storage
php artisan storage:link               # Crear enlace simbólico
```

11. Variables de entorno importantes

```
APP_NAME=GestionTienda
APP_ENV=local                # local | production
```

```
APP_DEBUG=true              # false en producción
APP_KEY=                    # Generada con php artisan key:generate
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=gestion_tienda
DB_USERNAME=root
DB_PASSWORD=

FILESYSTEM_DISK=public       # Para almacenamiento de imágenes
```

12. Despliegue en Railway paso a paso

Requisitos previos

- Cuenta en railway.app
- Repositorio en GitHub con el código

Pasos

1. **Crear proyecto en Railway**
    - New Project → Deploy from GitHub repo → selecciona tu repo
2. **Agregar MySQL**
    - En el proyecto → New → Database → MySQL
    - Railway genera automáticamente las variables MYSQL_HOST, MYSQL_PORT, etc.
3. **Crear nixpacks.toml** en la raíz del proyecto:

```
[phases.setup]
nixPkgs = ["php82", "php82Extensions.pdo", "php82Extensions.pdo_mysql",
"php82Extensions.mbstring", "php82Extensions.xml", "php82Extensions.curl",
"php82Extensions.zip", "nodejs_20", "npm"]

[phases.install]
cmds = ["composer install --no-dev --optimize-autoloader", "npm install", "npm run build"]

[phases.build]
cmds = ["php artisan config:cache", "php artisan route:cache", "php artisan view:cache"]
```

[start]
cmd = "php artisan migrate --force && php artisan db:seed --force && php artisan serve --host=0.0.0.0 --port=$PORT"

4. **Configurar variables en Railway**

    - APP_KEY → resultado de php artisan key:generate --show
    - APP_ENV → production
    - APP_DEBUG → false
    - DB_* → referencias a las variables de MySQL de Railway
5. **Push a main** → Railway despliega automáticamente

6. **Generar dominio**

    - En el servicio → Settings → Networking → Generate Domain

13. Solución de problemas comunes

Error	Causa probable	Solución
SQLSTATE: Connection refused	MySQL no está corriendo	Verificar que Laragon/MySQL esté activo
Class RoleSeeder not found	Autoload desactualizado	composer dump-autoload
419 CSRF token mismatch	Token expirado	Limpiar caché del navegador
403 Forbidden	Usuario sin permiso	Verificar rol asignado con $user->getRoleNames()
Imágenes no cargan	Falta enlace simbólico	php artisan storage:link
Unknown database	BD no creada	Crear la BD en HeidiSQL o MySQL